Transformer Feed-Forward Layers Are Key-Value Memories

(There is always a question in my mind when I am studying Transformers: what is the purpose of the feedfoward networks (FFNs)? Why do we need them after the mutilhead modules? How will it affect the result of prediction? So, I found this paper...)

- **Background concepts**
  - Key-Value Memory Network
    - Unlike the traditional NNs, Key-Value Memory Networks **explicitely** contain pairs of **memory cells** with a key and a value. Whenever there is an input, the network will calculate the *similarities* between the input, x, and each keys stored in the network as a weight to each key: the similar the key and input is, the higher weight will be produced. Then, the weight will be multiplied with values, and calculate the sum as the output.
    - Formally, we have:

    $p(k_i|x) \propto \exp(x \cdot k_i)$

    $MN(x) = \sum_i p(k_i|x)\ v_i$, where $p(\cdot)$ can be computed by softmax.

    Or, with matrices used:

    $$MN(x) = \text{softmax}(x \cdot K^\top) \cdot V \qquad (1)$$
    $$\text{Where:}$$
    $$K,\ V \in \mathbb{R}^{d_m \times d}$$

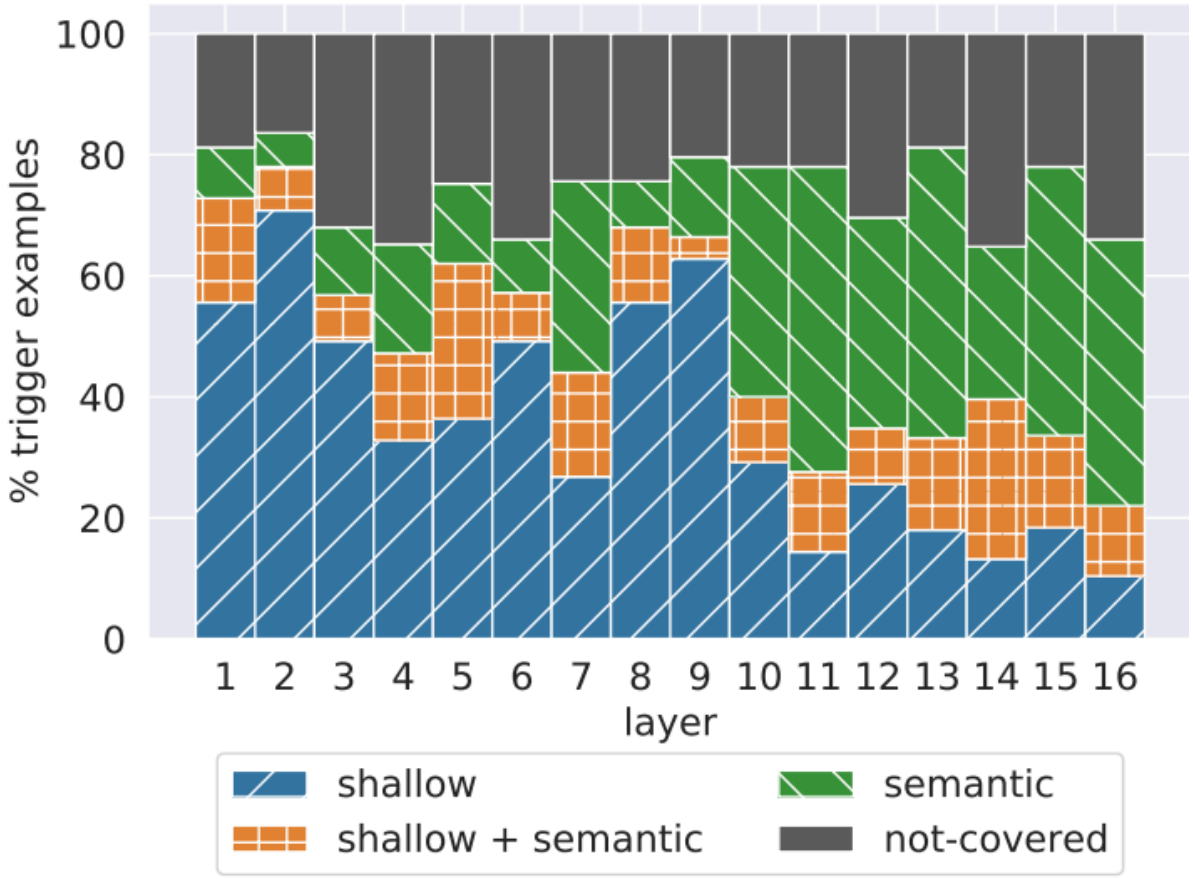
    - *Intuitively, this resembles **attention modules**, right?*
  - Feed-forward layers
    - Fully connected layers in the transform structures, normally after the **self-attention** modules, where we have:

    $$FF(x) = f(x \cdot K^\top) \cdot V, \qquad (2)$$
    $$\text{where:}$$
    $$K,\ V \in \mathbb{R}^{d_m \times d},$$
    $$f(\cdot) \text{ is a non-linear function}$$

    (Equation (2) is the version ignores the bias term; the full version is $FFN(x) = \sigma(xW_1 + b_1)W_2 + b_2$

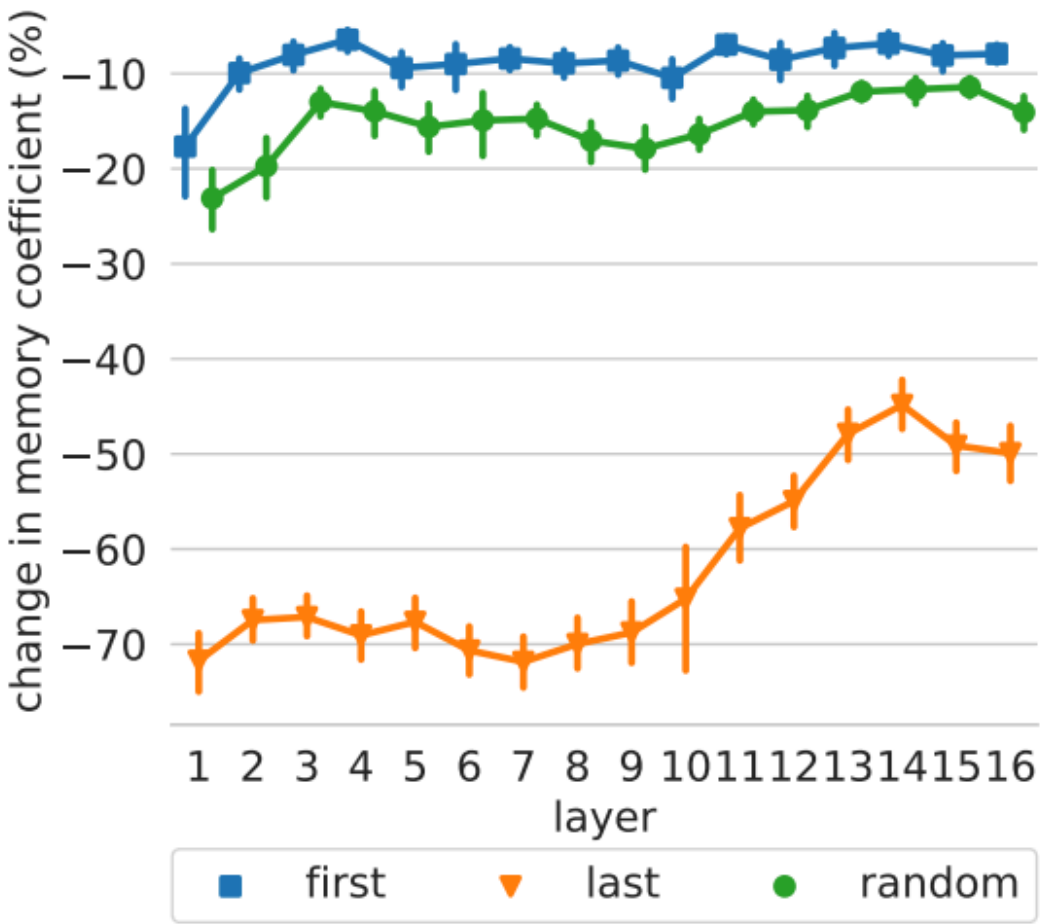
    - This part may have the major fraction of parameters in a transformer structure.
- **Feed-forward layers emulate neural memory**
  - We can easily found that equation (2) and equation (1) are similar. The main difference is that, while equation (1) uses a **softmax** to normalize the result of $x \cdot K^\top$, the FFN uses a **ReLU** function.
  - The researchers found that the dimensions of the K, Vmatrices in both cases are the same: both have $d$columns and $d_m$rows. Hence, they assumed that "**the hidden dimension dm is essentially the number of memories in the layer, and the activation $f(x \cdot K^\top)$, commonly referred to as the hidden layer, is a vector containing an unnormalized non-negative coefficient for each memory.**"

    - In the other word, the parameters of one row in K is one key, and one row in V is one value.
    - Then, it's time to verify our assumption that most parameters in FFNs are working for Key-Value memory.
- **Experiments -- keys**
    - Goal:
        - Is each key k_i related to a certain type of input (shallow and sematics)?
        - Are these keys responsible for shallow meaning, or sematics?
        - Is there a difference between the keys in shallow layers and deep layers?
    - The author used a pretrained transformer language model by Baevski & Auli (2019), with an input dimension of $d = 1024$ and a hidden dimension of 4096 (4096 neurons in total) per hidden layer, and 16 hidden layers -- in total, there are 65536 neurons, or, 65536 keys. In this experiment, only 160 keys will be analyzed with 10 random keys per layer.
    - Then, the researchers will retreiving the trigger examples. For a certain k_i in layer l, we compute the memory coefficients $\mathrm{ReLU}(x_j^l \; k_i^l)$, where x_j is the prefix of sentences in the WikiText103's training set (for example, sentence "I eat apples", the prefixes are "I", "I eat", "I eat apples"), and find out the prefix such that it can produce the **highest score** for k_i^l, which is called one of the *top-t trigger examples***.** In this experiment, $t = 25$: we choose 25 such triggers to analyze for each key.
    - The next step is Pattern Analysis. Human "experts" (aka NLP graduate students) are invited to:
        - Identify repetitive patterns that occur in at least 3 prefixes
        - Describe each pattern
        - Classify each recognized pattern as "shallow" (e.g. recurring n-grams) or "semantic" (recurring topic).
    - Result:
        - **Memories are associated with human recognizable patterns.** For every key, the "experts" can find at least one patterns, with 3.6 patterns in average per key. Researchers also found that most (65%-80%) retrieved prefixes associated with one pattern. As shown in the figure below:



- Shallow layers tend to follow shallow patterns (n-gram recurrance), and the deep layers tend to find semantic patterns. *From the figure above, it can be seen that before layer 9, shallow pattern occupies a huge % of examples; after layer 10, however, semantic patterns are generally found.*
- To further prove the findings above, the researchers sample 1600 random keys (100 keys per layer) and apply local modifications to the top-50 trigger examples of every key, and then delete either the first, the last, or the random token from the sentence and check the change in scores.
    - It is found that removing the first and the random token **doesn't** produce significant changes; removing the last token causes great changes in the scores in shallow layers, but has no significant effect for deeper layers. **This coincides with the findings above.**

  (N-grams check the previous n-1 words to find the pattern to predict the next word; removing the last word may change hugely in pattern finding, so the scores will also change greatly; semantic is on a sentence-wise scope: the model attends the meaning of the whole sentence. Removing one word won't change score much.)

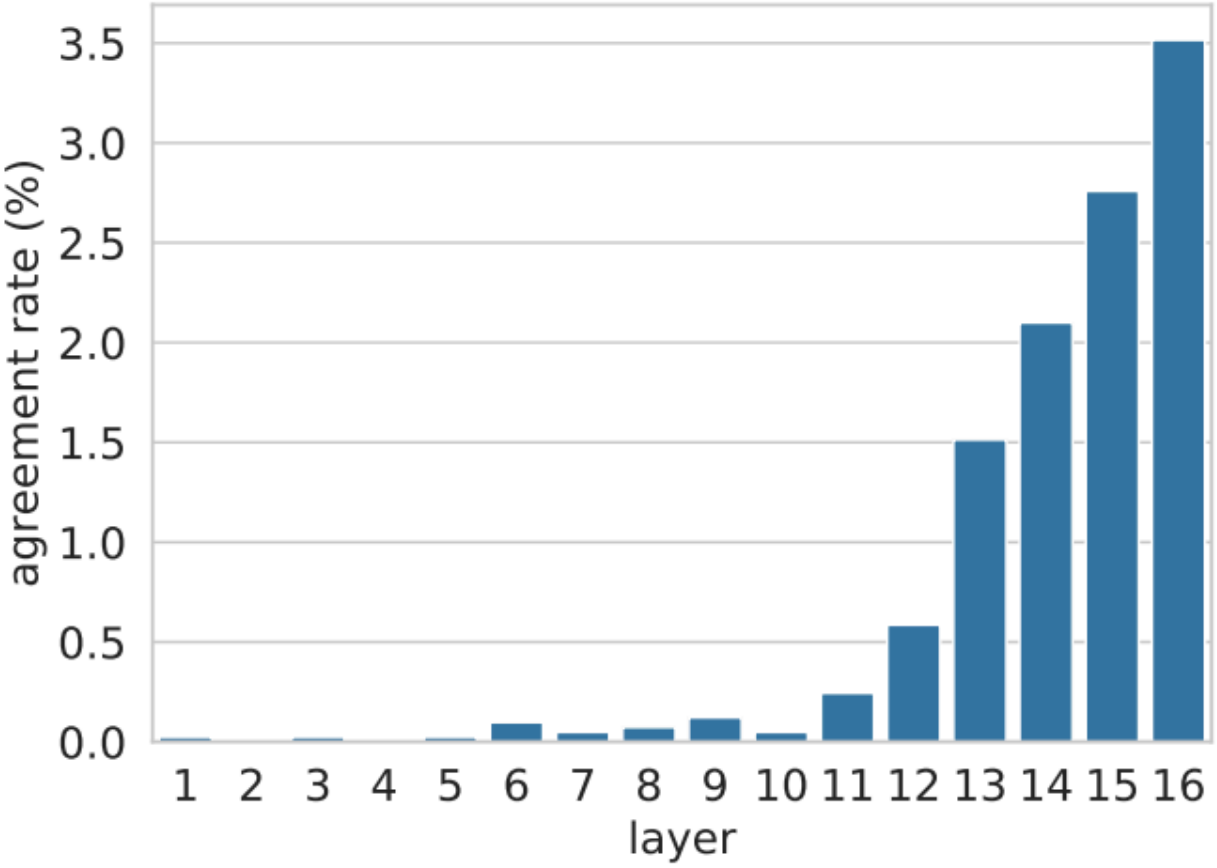


- **Experiment -- Values**
  - Goal: show the infomation stored in the values; show that the values can be regarded as the distribution over the output vocabulary.
  - Firstly, the researchers cast the values to probability by multipliying the value vectors with the output embedding matrix (*The matrix to project the **last** hidden state to logits,* $\text{Logits} = h_L \cdot E$) and softmax-ing it to probabilities. Formally:

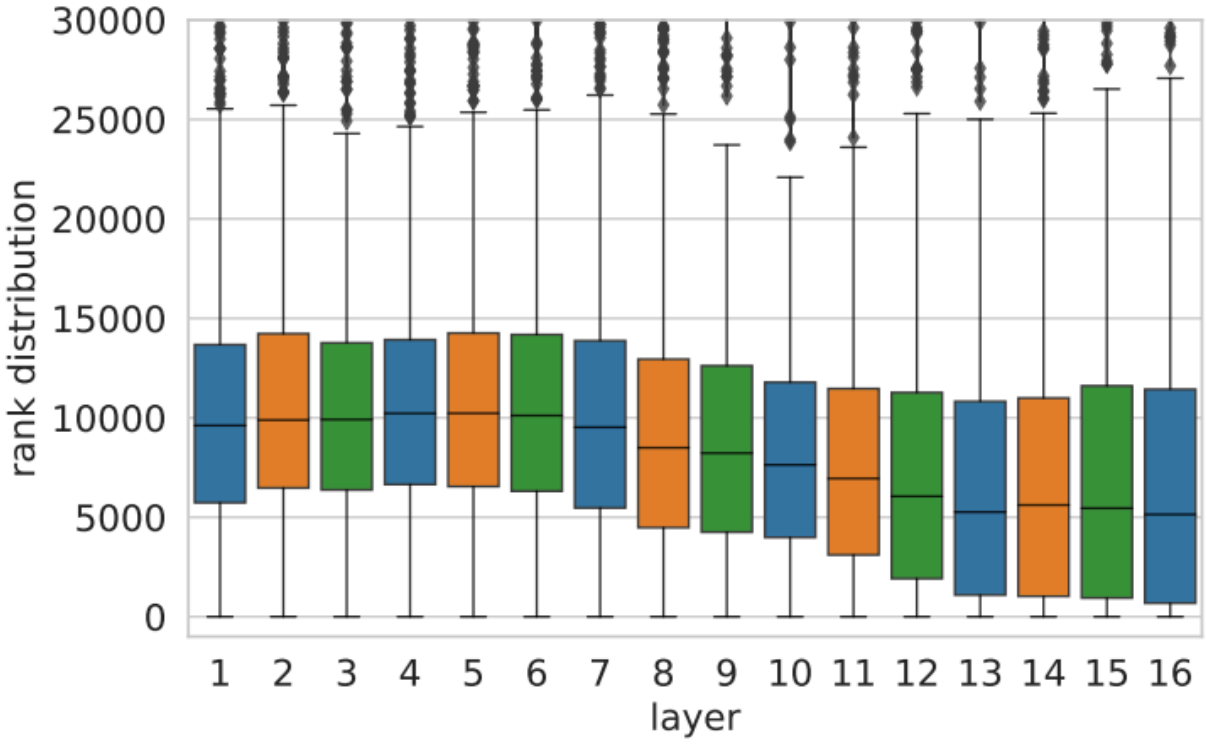
    $$p_i^l = \text{softmax}(v_i^l \cdot E),$$
    $$\text{where:}$$
    $$E \in \mathbb{R}^{d \times |\text{vocab}|}$$

    Note that normally, we use v_i^l in this way: $\text{FFN output} = \text{ReLU}(x \cdot k_i) \cdot v_i = m_i \cdot v_i$, so the way researchers use to produce probabilty, p_i^l is not "calibrated", so this should not be the **[final]** output. However, in this experiment, we only care about **[ranking]**, which will **not** affected by the weight produced by ReLU. That is, if the top-ranked token based on distribution v_i^l is used, then, mathematically, it can be safely said that $\arg\max_{v_i^l}(a \cdot z) = \arg\max_{v_i^l}(z)$.

    And, of course, we do use the token with the top-rank. Specifically, the top-ranked token will be compared to the next token, w_i^l, of the **[top-1 trigger]** prefix from the last part. It is defined that $\text{agreement rate} = \frac{\text{number of } (w_i^l = \arg\max(p_i^l))}{\text{total number of k-val pairs}}$.
  - Findings: Based on the figure below, it can be seen that with layer becomes deeper, the agreement rate increases from layer 11 and reaches the maximum of 3.5% in the deepest layer. This value matters as these results are significantly higher than the agreement rate produced by a randomly select token.

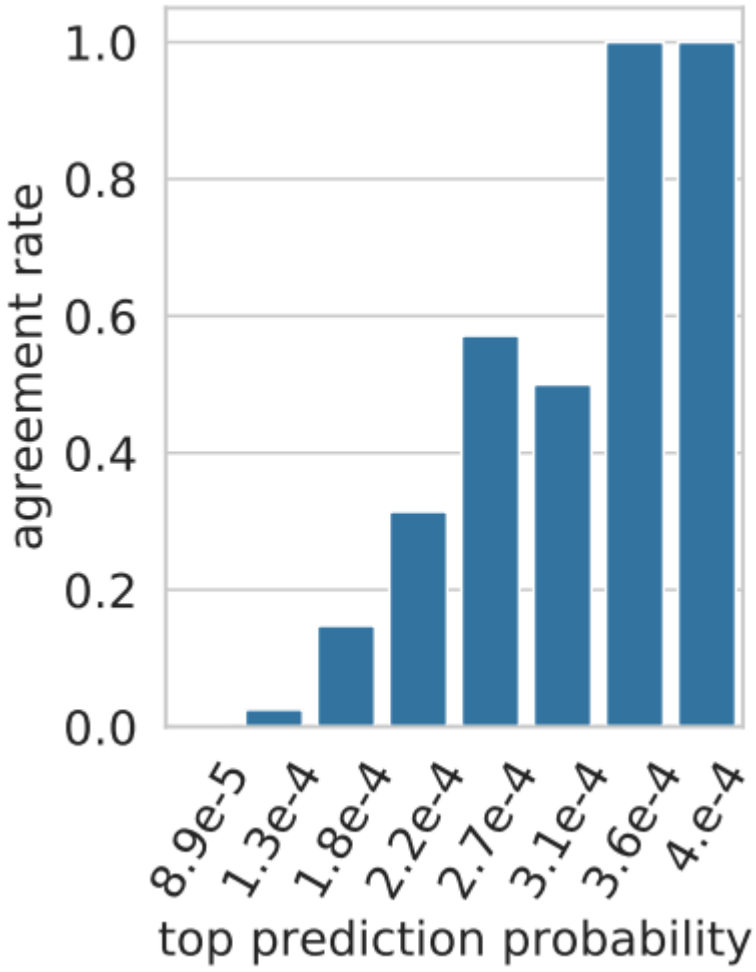


- Then, the researchers obtained the rankings of w_i^lgiven the distribution generated by v_i^l. They found something similar: the deeper the layer is, the higher the rank of w_i^lwill be, as shown below.



- Then, the researchers explore the relationship between agreement rate and the maximum

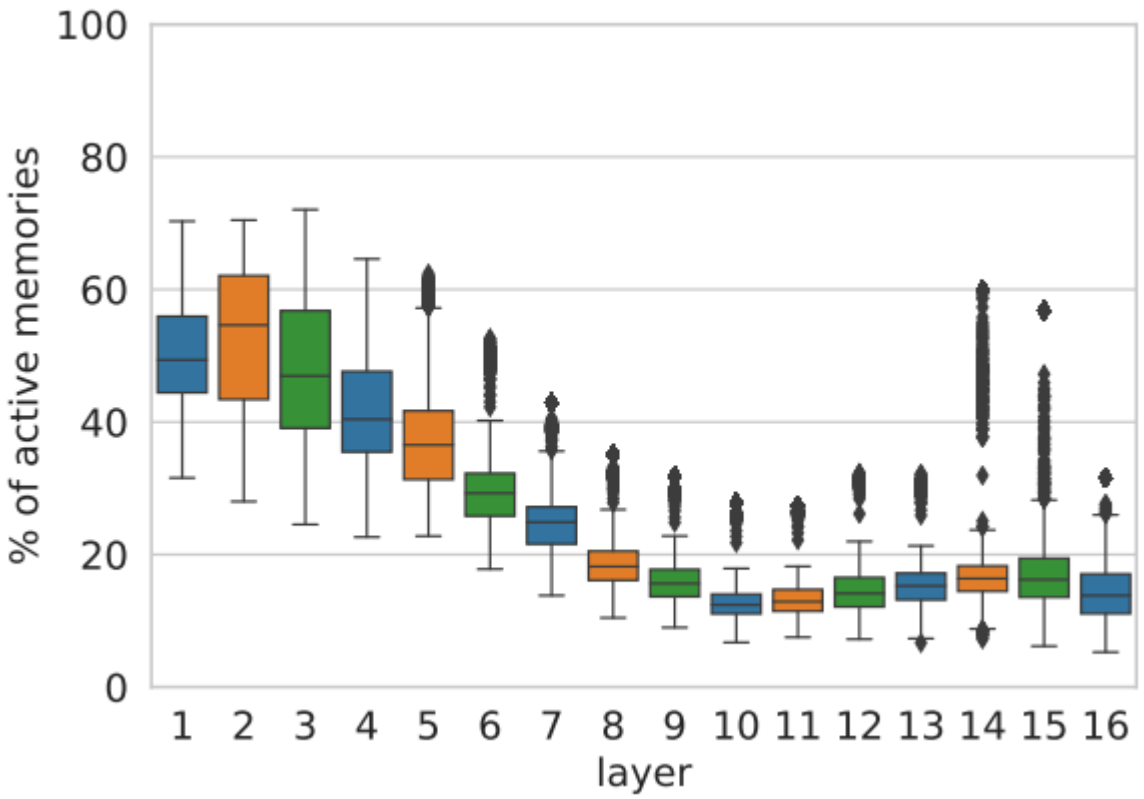
probability from each distribution p_i^l. They find that the higher $\max p_i^l$ is, or, the confident the value is, the higher agreement rate will be, as shown below.



- Based on the experiments above, the researchers conclude that the deeper layers tend to directly store the information about how to predict the output, but the shallower layers have no property. One possiblity is that the lower layers don't use the output embedding E to project the value vector. Interestingly, SOME of the intermediate layers operate in the similar or the same output embedding space as the deeper layers, which should be explored further.

- **Experiment -- Aggregating memories**
  - In this section, the authors explore how the decision is made in a single layer and across multiple layers.
  - Part 1: Intra-layer exploration
    - The researchers firstly test the intra-layer behavior of the model to explore whether the final decision is dominated by one memory cell or is a composition of multiple cells, given that the output of each layer:

      $\mathbf{y}^l = \sum_i (\mathbf{x}^l \cdot \mathbf{k}_i^l) \cdot \mathbf{v}_i^l + \mathbf{b}^l$
    - Firstly, they measured the number of activated memories (neuron with value >0) using the data from **validation sets** as the researchers are interested in the inference stage of the model. The researchers found that though decreasing as the layer becomes deeper, the number of active memory cells is around several hundred to thousand (10% - 50% of total cells), as shown below.



- The researchers, then, count the number of instances where the feed-forward layer's top prediction is **different** from all of the memories' top predictions. That is, they project the value vector v_i^l to the possibility distribution using the previous equation.

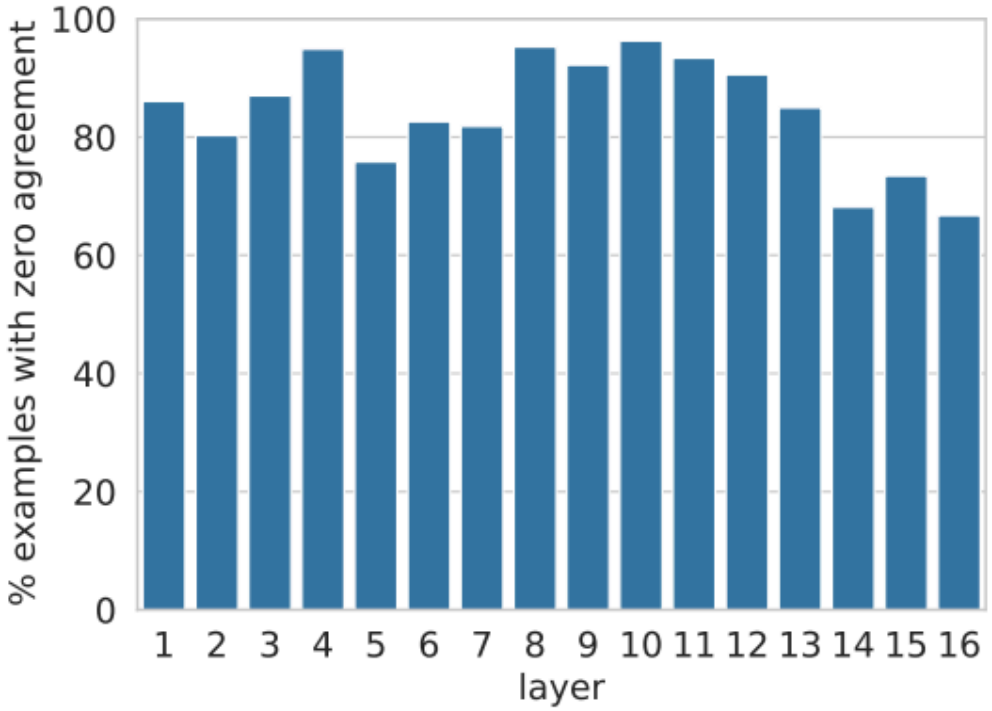
  $$\text{top}(\mathbf{h}) = \arg\max(\mathbf{h} \cdot E)$$

  Then, they compute the number of examples where,

  $$\forall i : \text{top}(v_i^l) \neq \text{top}(y^l),$$

  meaning that the layer output is **different** from **ALL** neurons' (memory cells') output (and thus layer output is a combination of each neuron's output).

  As a result, the authors find that for most layers, the layer output is different from the output of each neuron, with a fraction of at least ~68%, as shown below.



For those cases where $\exists i : \text{top}(v_i^l) = \text{top}(y^l)$, the researchers conclude that 60% of these cases include a common stop word vocabulary, such as "the" and "a", and 43% of these cases have an input prefix with less than 5 words, suggesting that very shallow (or, common) patterns are *cached* in the key-val memory cells and hence don't need compositionality.

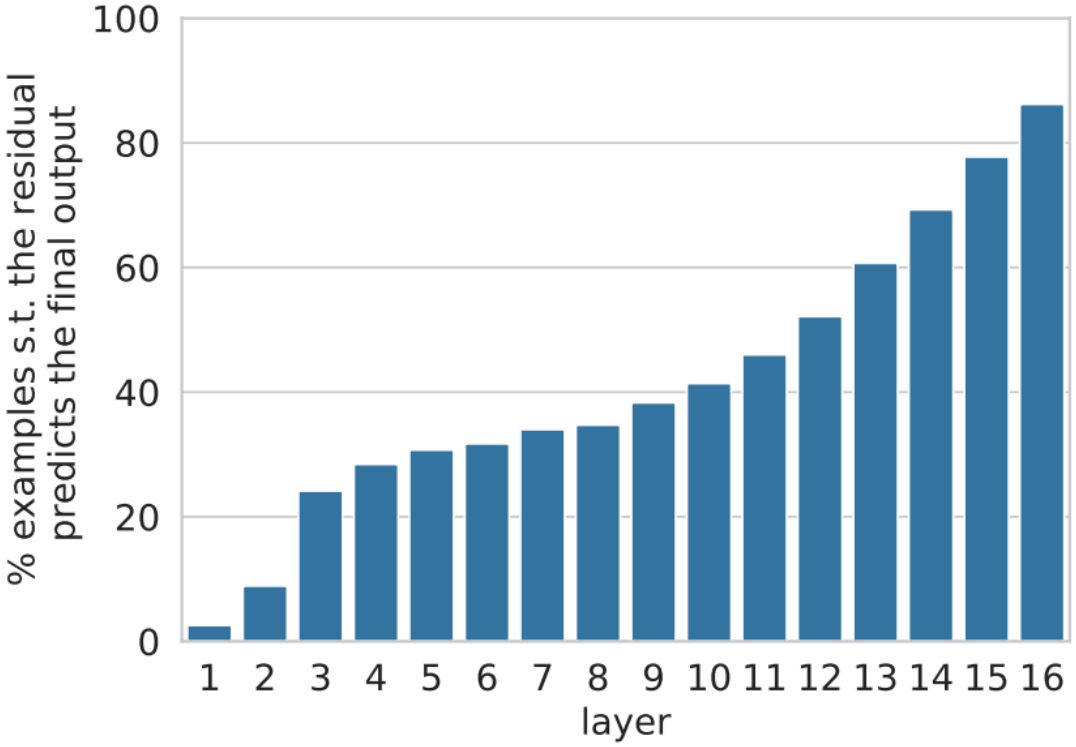
- Part 2: Inter-layer exploration
  - The layers in FFN are connected using residual connection, that is, add an original copy of input to the output of that layer, which is detailedly illustrated here. In the paper, the researchers use the following version:

    $$\mathbf{x}^l = \text{LayerNorm}(\mathbf{r}^l)$$
    $$\mathbf{y}^l = \text{FF}(\mathbf{x}^l)$$
    $$\mathbf{o}^l = \mathbf{y}^l + \mathbf{r}^l$$

    The researchers assume that the sequential connection is actually a process of **refinement**, that is, the prediction will be determined in the shallower layers, and the following layers are only for refine the prediction. Thus, they measured the proportion of cases where the input $\mathbf{r}^l$ is the same as the output $\mathbf{o}^l$. Formally,

    $$\text{top}(\mathbf{r}^l) = \text{top}(\mathbf{o}^l)$$

    They found that ~1/3 of the model's predictions are determined in the shallower layers; the percentage increases rapidly after layer 10, implying that the vast number of "hard" decisions are done before the final layer, as shown below.



- The experiment above only answers a part of the question that the decision is made in the shallower layers, but is this a process of refinement? To solve this question, the researchers firstly find out the final predicted **token** w:

  $$w = \text{top}(\mathbf{o}^l)$$

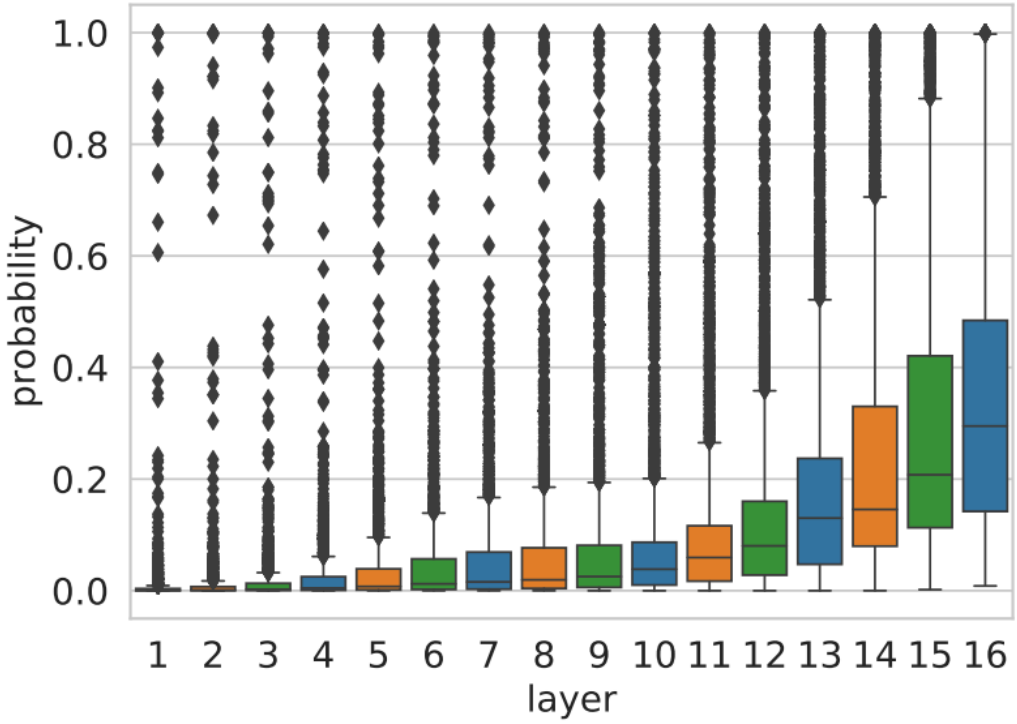
  Then, cast $\mathbf{r}^l$to probability distribution by (similar to the trick played for v_i^l):

  $$\mathbf{p} = \text{softmax}(\mathbf{r}^l \cdot E)$$

  Lastly, find out the corresponding probability of $w$in $\mathbf{p}$:

  $$p = \mathbf{p}[w]$$

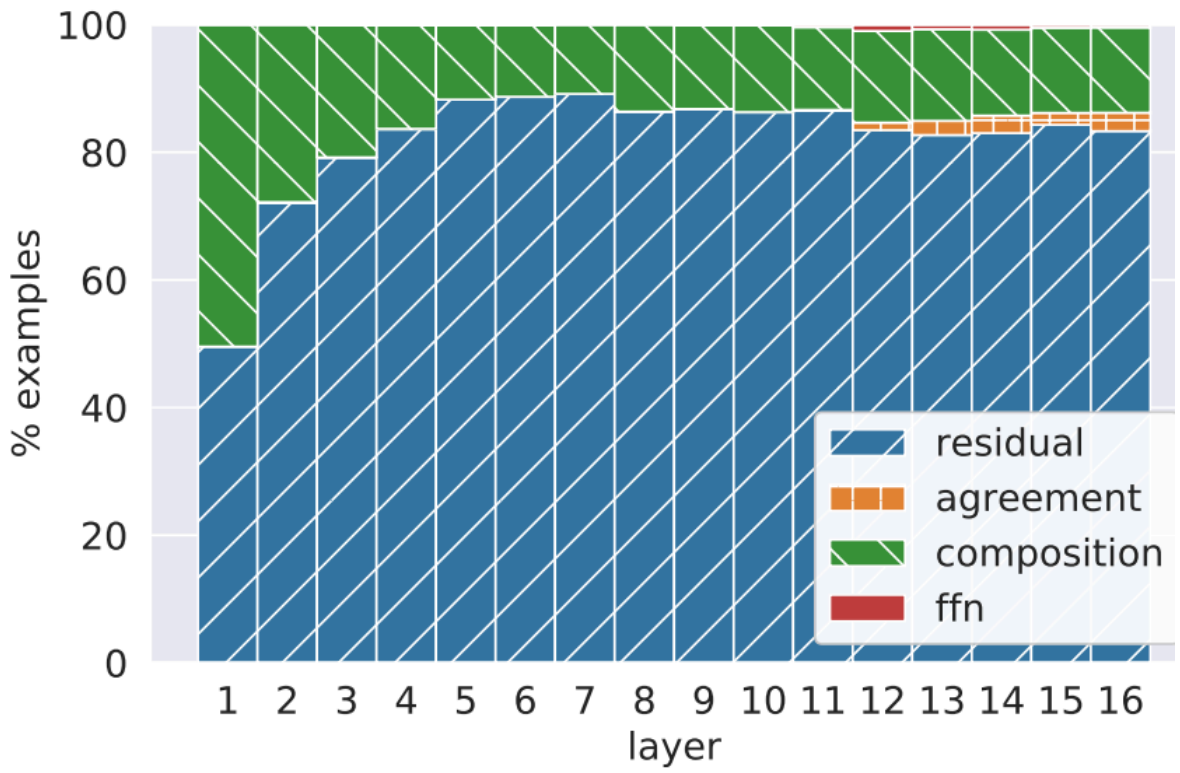
  In this way, the higher $p$is, the more confident the model is to its prediction. Then, they get this:



From the figure above, it can be seen that it **is** refining the prediction: the deeper = the more confident.

- Then, what is the relationship between the output of FFN and the residual? What is the relationship between the final output and these two? To solve these questions, researchers create four cases
  - FFN's output dominates (but $\neq$residual)
  - Residual dominates (but $\neq$FFN)
  - Final output = FFN = Residual (agreement)
  - Final output $\neq$ FFN and $\neq$Residual (composition)

The result is shown below:



In most cases, the final output is the same as the prediction of the residual vector (residual+agreement). When the residual prediction is overruled, the top-1 of the FFN is rarely used directly; more often there is a **composition**: i.e., the final output is another candidate for the fusion of the residuals with the FFN distribution.

```
The paper also mentions that the FFN's result will not be directly applied to the final result; instead, it is like a veto with elimination mechanism: if any layers don't agree with the residual, it will decrease the weight of the residual answer and increase what they think is proper, which can explain why a number of cases are composition: The final output is neither the top-1 of the residuals nor the top-1 of the FFN, but the new first name that emerges from the fusion of the two distributions.

* Plus, the researchers conclude that even at the last layer, **the model is still able to change the result**. Among 100 random cases of last-layer composition, 66 cases changed the meaning of the prediction significantly, and in the 34 examples, the last layer shifted the residual prediction to a related word.&#x20;
```

- **Conclusion**
  - The FFN module of transformers can be regarded as key-value memory network.
  - Keys are related to human-interpretable patterns;
  - The keys of **deep layers** induce distributions over the output vocabulary that correlate with the next-token distribution of patterns in the corresponding key;
  - The output is formed via **aggregation** of the distributions. They are first composed to form

individual layer outputs, which are then refined throughout the model's layers using residual connections.